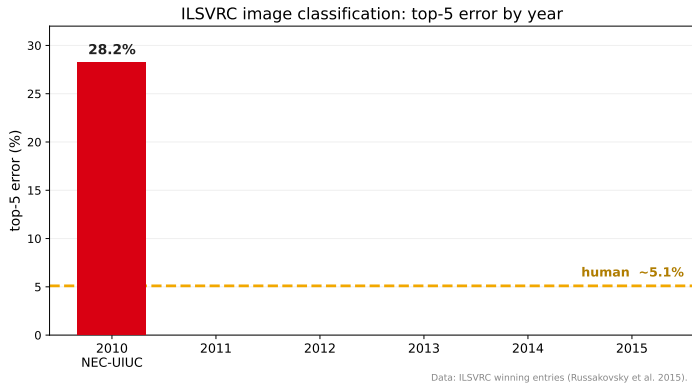


Lecture 17: CNN Architectures

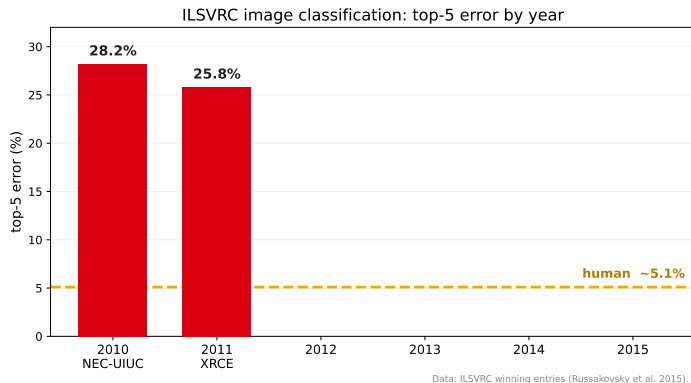
Ten years that changed computer vision

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

One chart, one question

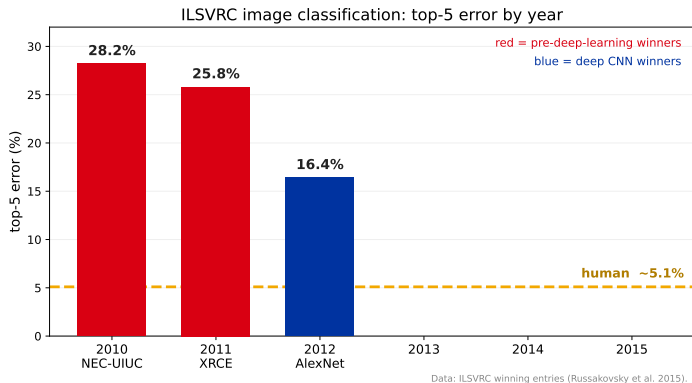


One chart, one question



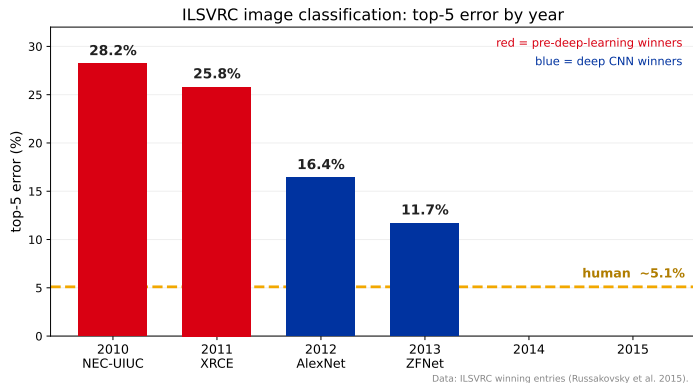
Predict: something happened in 2012. What?

One chart, one question



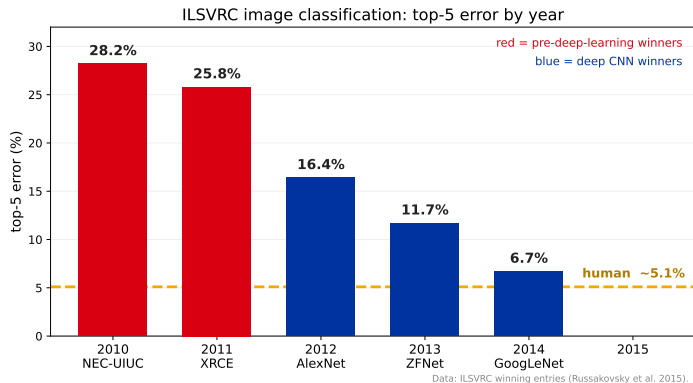
AlexNet - the first deep CNN winner - cut the error from 26% to 16% in one year.

One chart, one question



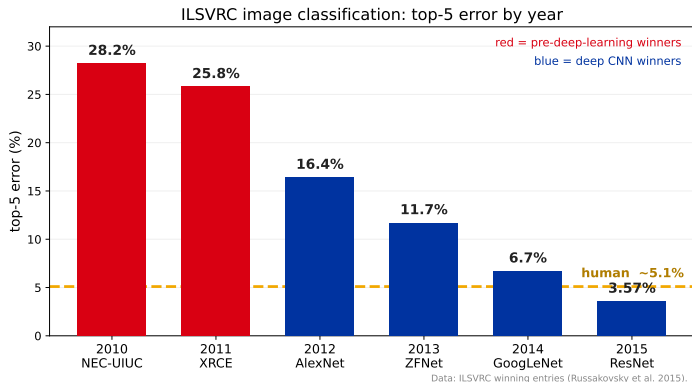
AlexNet - the first deep CNN winner - cut the error from 26% to 16% in one year.

One chart, one question



AlexNet - the first deep CNN winner - cut the error from 26% to 16% in one year.

One chart, one question



By 2015, machines passed the human baseline. **Everything in this lecture is on this chart** - the benchmark, the architectures, and the people who built them.

Outline

The benchmark that made deep learning

The classics: LeNet, AlexNet, VGG

Going deeper: degradation and ResNet

The wider zoo

Using architectures in practice

Epilogue: the pioneers and the field, 2026

ImageNet

Before the architectures, the arena: a dataset so large it changed what was worth building.



The bet on data

In 2006, while most of computer vision tuned **algorithms**, **Fei-Fei Li** bet that what the field actually lacked was **data**.

- ▶ Built on **WordNet**'s noun hierarchy - with WordNet's creator, Christiane Fellbaum.
- ▶ Images scraped from the web; labeled by **Amazon Mechanical Turk** workers.
- ▶ Presented at CVPR **2009**, to little fanfare: **~14M labeled images**, ~22,000 categories.

The lesson that outlives every architecture in this deck: **data bets age better than algorithm bets**. Models come and go; ImageNet made all of them possible.

ILSVRC and top-5 error

The annual challenge (ILSVRC) used a subset: **1.2M training images, 1,000 classes**.

Top-5 error: the model proposes five labels; it is correct if the true label is among them - fair scoring when an image contains several things.

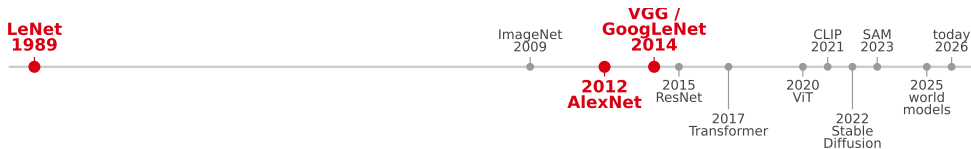
A shared benchmark accelerates a field for the same reason we always compare models on *held-out* data ([06]): everyone measures the same thing, so progress is real and comparable.



Real ILSVRC test images with a model's five guesses.

Three architectures, three ideas

The blueprint (LeNet), the breakout (AlexNet), and depth done simply (VGG).



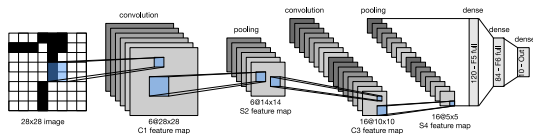
LeNet (1998): the blueprint

Yann LeCun, AT&T Bell Labs (joined 1988): backprop-trained convolutional nets reading **handwritten digits**.

conv-pool-conv-pool-dense:
everything in L16 was already here.

Deployed commercially by NCR in **check-reading machines**; at its peak the system read an estimated **10-20% of all checks** written in the US.

Then the field moved on to SVMs, and CNNs went quiet for a decade. What was missing: **compute** and **data**.



Keep an eye on LeCun - he returns at the end of this lecture.

► play with a LeNet in 3D (Adam Harley)

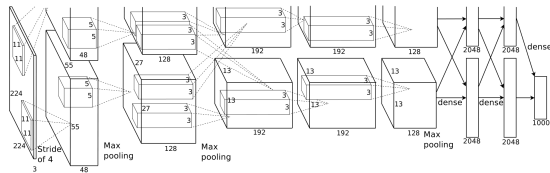
► CNN Explainer

AlexNet (2012): the breakout

Same blueprint, **scaled**: 8 layers, **60M parameters** - too big for one GPU, so Krizhevsky **split the network across two** consumer gaming cards (GTX 580, 3GB each) and trained **5-6 days** (~90 epochs over 1.2M images) on a single PC.

The GPU was the unlock: L14's matrix-multiply argument, made history.

Authors: Krizhevsky, Sutskever, Hinton (2012).



The paper's own diagram: one GPU runs the top half, one the bottom.

The "new" tricks are all **L15 material deployed at once**: ReLU (vs saturating sigmoids), dropout, data augmentation. You already know every ingredient - they had the hardware to use them at scale.

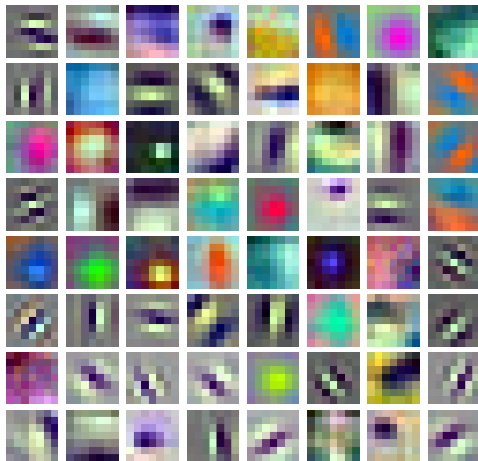
What AlexNet-style networks learn

First-layer filters of a modern pretrained network (ResNet-18, trained on ImageNet): crisp **Gabor-like edge, blob and color detectors**.

The **L16 kernel zoo** - reinvented by gradient descent, designed by nobody.

L16's tiny Fashion-MNIST kernels looked blurry; at ImageNet scale the learned kernels become *textbook*. The hand-designed → learned arc closes here.

ResNet-18 first-layer filters, learned on ImageNet (64 x 7x7 RGB)



VGG (2014): depth via small filters

Only 3×3 **convolutions**, stacked. Two stacked 3×3 layers see a 5×5 receptive field - so which costs fewer weights?

Take a minute and compute it for $C = 64$ channels in and out (ignore biases):

$$\text{two } 3 \times 3 \text{ layers: } 2 \times 9C^2 \quad \text{vs} \quad \text{one } 5 \times 5 \text{ layer: } 25C^2$$

VGG (2014): depth via small filters

Only 3×3 **convolutions**, stacked. Two stacked 3×3 layers see a 5×5 receptive field - so which costs fewer weights?

Take a minute and compute it for $C = 64$ channels in and out (ignore biases):

$$\text{two } 3 \times 3 \text{ layers: } 2 \times 9C^2 \quad \text{vs} \quad \text{one } 5 \times 5 \text{ layer: } 25C^2$$

$$2 \times 9 \times 64^2 = \mathbf{73,728} \quad \text{vs} \quad 25 \times 64^2 = \mathbf{102,400}$$

Deeper **and** cheaper - with an extra ReLU between the two layers **for free**. That one observation is the whole VGG design: repeat 3×3 blocks, double the channels after each pool.

Who built VGG

Karen Simonyan and Andrew Zisserman, Oxford's **V**isual **G**eometry **G**roup - and yes, Karen Simonyan is an Armenian name.

His arc since:

- ▶ **DeepMind** (2014-2022) - contributor to AlphaZero, AlphaFold, WaveNet;
- ▶ co-founder of **Inflection AI** (2022, with Reid Hoffman and Mustafa Suleyman);
- ▶ now **Chief Scientist at Microsoft AI**.

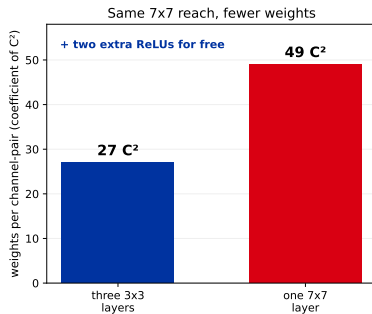
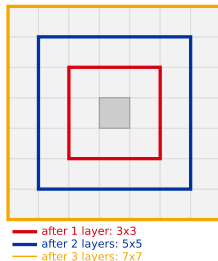
A VGG author's career *is* the last decade of AI in miniature: benchmark vision → deep-learning lab → startup → big-tech frontier lab.

Receptive field: how far a neuron sees

The **receptive field** of a neuron is the input region that can influence it. Stack conv layers and it **grows with depth**: three 3×3 layers reach 7×7 .

Deep layers see *objects*; early layers see *edges*. That is the feature hierarchy of L16, measured in pixels.

Receptive field grows with depth
(stacked 3×3 convolutions)



Growing the receptive field *without* paying for depth is also possible - dilated convolutions, coming in L19.

If depth is good, why not 100 layers?

Because past ~ 20 layers, plain networks get *worse* - and the fix reshaped every deep network since.

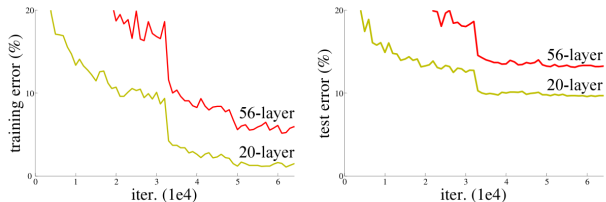


Predict first: the degradation problem

A **56-layer** plain net vs a **20-layer** plain net - same data, same recipe. **Which one has lower training error?**

Predict first: the degradation problem

A **56-layer** plain net vs a **20-layer** plain net - same data, same recipe. **Which one has lower training error?**



The 20-layer one (He et al. 2015, Figure 1). This is *not* overfitting - it is *training* error. Deep plain nets are simply **hard to optimize**: the deeper model could imitate the shallow one by making 36 layers do nothing, yet gradient descent fails to find even that.

The residual idea

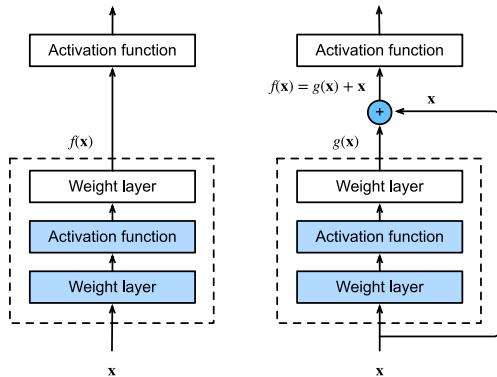
Let the block learn the **correction**, not the whole mapping:

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

The **skip connection** carries $\mathbf{x}$ for free; the layers only learn the residual $F(\mathbf{x})$.

Learning "do nothing" is now trivial - push $F \rightarrow 0$ - so **extra layers can no longer hurt**.

If dimensions change, project the shortcut:
 $H(\mathbf{x}) = F(\mathbf{x}) + W_s \mathbf{x}$.



Left: a plain block must learn all of $f(\mathbf{x})$. Right: a residual block only learns the difference $g(\mathbf{x})$.

You have seen residual learning before

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

Recall: where in this course have you already met a model that only fits a **correction** to what already exists?

You have seen residual learning before

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

Recall: where in this course have you already met a model that only fits a **correction** to what already exists?

Gradient boosting (ch4): every new tree fits the *residual* of the ensemble so far. A ResNet block does the same thing **in depth** - boosting stacks residual learners in sequence, ResNet stacks them layer on layer.

Analogy, not identity: boosting fits each learner to the *frozen* ensemble's residual, stage by stage; ResNet blocks train *jointly*, end to end.

Why skips also fix gradients

The identity path is not just easy to learn - it is an **undamped route for the gradient** to flow backwards:

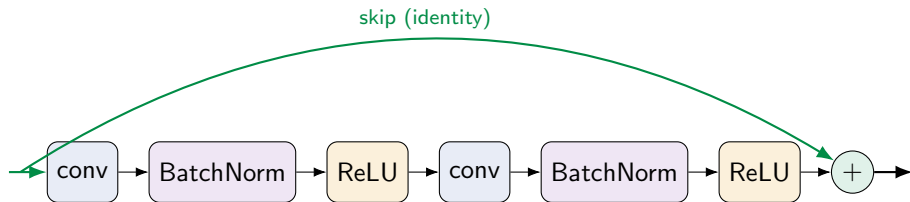
$$\frac{\partial H}{\partial \mathbf{x}} = \frac{\partial F}{\partial \mathbf{x}} + \mathbf{I}$$

Even if the block's own gradient $\partial F / \partial \mathbf{x}$ is tiny, the $\mathbf{I}$ term carries the signal straight through - across a hundred layers if needed.

The **vanishing-gradient** problem from L15, relieved by *wiring* rather than by activation choice. Same enemy, a structural weapon.

The modern conv block

What actually repeats inside a modern CNN:

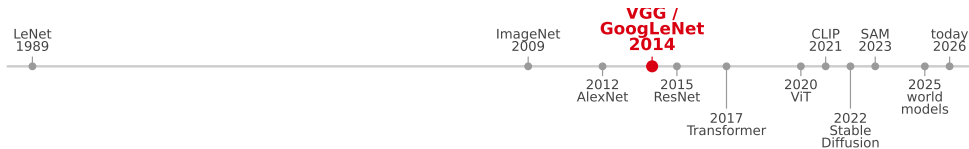


conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU, repeated, plus skip connections.

BatchNorm is the **L15 formula, unchanged** - normalize per batch, then scale and shift with learned γ, β . Nothing new to learn today.

Other tricks that stuck

Three ideas beyond depth - and one network built entirely from them.



1×1 convolutions

A 1×1 conv is a **per-pixel dense layer across channels**: it mixes *depth* without touching *space*.

Two jobs:

1. **cheap channel reduction** before expensive convolutions,
2. an extra **nonlinearity** almost for free.

Reduce 256 $\rightarrow$ 64 channels:

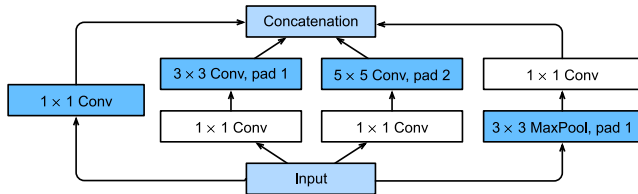
$$1 \times 1 : 256 \cdot 64 + 64 = \mathbf{16,448} \quad \text{vs} \quad 3 \times 3 : 9 \cdot 256 \cdot 64 + 64 = \mathbf{147,520}$$

Pointwise convolutions return in L19 (separable convolutions - how phones afford CNNs).

The Inception block

Why choose a kernel size at all?
Run 1×1 , 3×3 , 5×5 **and** a pooling path **in parallel**, and **concatenate** the maps.

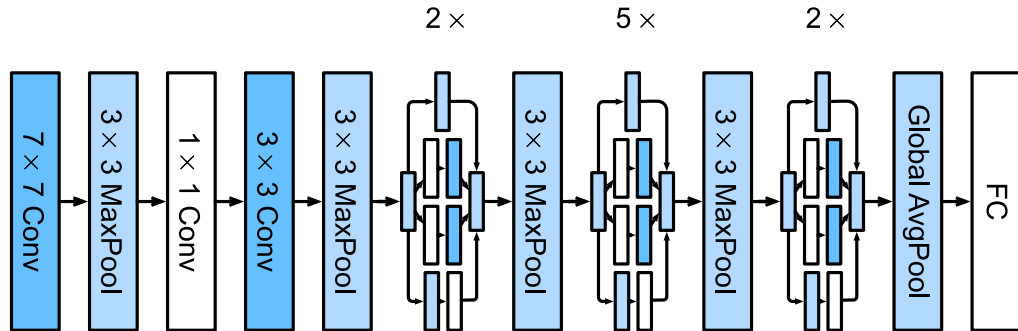
The network picks its own mix, layer by layer.



The 1×1 bottlenecks (white boxes) are what keep four parallel paths *affordable* - the previous slide's trick, deployed.

GoogLeNet: a network of Inception blocks

The full **2014 winner**: ~ 22 layers of stacked Inception blocks - won ILSVRC 2014 at **6.7%** top-5 error.



Under **7M parameters** - against AlexNet's **60M**, with far better accuracy. The zoo tricks *compound*. And note how it ends: no giant dense head, but a *global average pool* - next slides.

The dense-head problem

Predict first: AlexNet has 60M parameters. Where do *most* of them live - the conv layers, or somewhere else?

The dense-head problem

Predict first: AlexNet has 60M parameters. Where do *most* of them live - the conv layers, or somewhere else?

In the **flatten** → **dense head**. AlexNet's dense layers hold roughly **58 of its 60M** weights.

Why: flattening keeps every pixel of every map. Take a modest final stack of **256 maps of** 100×100 , into a 10-class dense layer:

$$256 \cdot 100 \cdot 100 \cdot 10 = \mathbf{25,600,000} \text{ weights}$$

More than all the conv layers combined - spent on one classification layer.

Global average pooling

The fix (from Network-in-Network): **average each final map to one number.** 256 maps $\rightarrow$ a 256-vector $\rightarrow$ dense-10.

Compute the head's weight count yourself.

Global average pooling

The fix (from Network-in-Network): **average each final map to one number**. 256 maps $\rightarrow$ a 256-vector $\rightarrow$ dense-10.

Compute the head's weight count yourself.

$$256 \cdot 10 = \mathbf{2,560} \text{ weights} \quad (\text{was } 25,600,000) - \text{a } \mathbf{10,000\times} \text{ cut}$$

Bonus: GAP acts as a **regularizer** (no giant head to overfit) and accepts **any input size** (the average does not care how big the map is).

The **L16 pooling** operation with a *global* window. Used by everything modern - ResNet already ends this way.

Grouped convolution

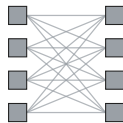
Split the input channels into g **groups**; each group gets its own filters, with no mixing across groups. Parameters and compute drop by a factor of g .

- ▶ You already saw it: **AlexNet's** two-GPU split *was* a grouped convolution ($g = 2$) - one group per card.
- ▶ Push it to the extreme, **one group per channel** ($g = C$), and it is the **depthwise** convolution behind separable convs (L19).
- ▶ **ResNeXt** (2017) made g a design knob ("cardinality") - a third lever beside depth and width.

Grouped convolution: fewer cross-channel connections -> $g \times$ fewer weights

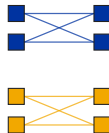
Standard: every output sees every channel Grouped ($g = 2$): each output sees only its group

input channels output channels



16 connections

input channels output channels



8 connections

$g=2$ (AlexNet) to $g=C$ (depthwise): one operation, two extremes. *Deformable conv* (Dai et al., 2017) goes further - it learns *where* to sample.

Scaling and searching

Where architecture design went next - one line each:

- ▶ **DenseNet**: like ResNet, but *concatenate* the skip instead of adding it.
- ▶ **EfficientNet**: scale depth, width and resolution *together* with one compound coefficient.
- ▶ **Neural architecture search**: architectures are now *found* by search as much as designed by hand.

None of these change the vocabulary you now have: conv blocks, skips, 1×1 s, GAP. The d2l.ai chapter on modern CNNs covers each, for the curious.

Nobody types out a ResNet

Every architecture in this lecture is one import away - trained weights included.



torchvision.models

```
import torch
from torchvision import models

net = models.resnet18(weights="IMAGENET1K_V1") # pretrained
print(sum(p.numel() for p in net.parameters())) # 11,689,512

net.fc = torch.nn.Linear(512, 3) # new head: export/market/juice
```

Load an architecture, inspect it, swap the head for *your* classes - three lines each.

What "keep the body, replace the head" buys you - and when to also retrain the body - is the whole next lecture.

► [torchvision models docs](#)

Which architecture, when

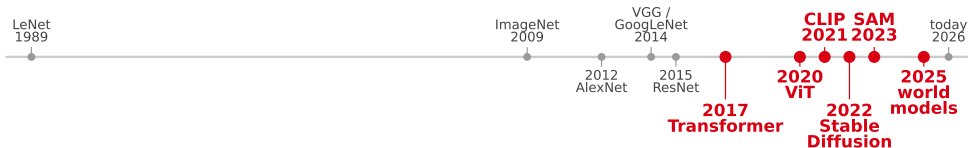
Back to the **pomegranate sorting line**:

- ▶ A Fashion-MNIST-sized toy problem (small grayscale crops, few classes) → the **small custom CNN from L16**: 9k parameters, trains in minutes, easy to debug.
- ▶ The real sorting camera (224×224 color photos, subtle grade differences) → start from a **pretrained ResNet-family model** and adapt it.

Adapting a pretrained model to your data *is* **transfer learning** - exactly where the next lecture picks up.

Where did everyone go?

The people on this timeline are still building - and still disagreeing about what comes next.



The pioneers today

2018 **Turing Award** to Hinton, LeCun and Bengio - "for deep learning".

And the LeNet hook resolves. After ~ 12 years as Meta's chief AI scientist, **Yann LeCun left** (Nov 2025) and founded **AMI Labs** (Paris), raising about **\$1B within months** - to bet *against* large language models and on **world models** (JEPA): systems that learn how the physical world works by predicting in *representation space*, not in pixels or tokens.

The man who made CNNs work thinks the next leap is **not a bigger chatbot**. The field's pioneers still disagree about its future - that is healthy, not alarming.

Sources: CNBC, Nov 2025; MIT Technology Review, Jan 2026.

One paper, three fates

The three names on the AlexNet paper, thirteen years on:

- ▶ **Geoffrey Hinton** - 2018 Turing Award, then the **2024 Nobel Prize in Physics** (with Hopfield); now the field's loudest safety voice.
- ▶ **Ilya Sutskever** - co-founded **OpenAI**, chief scientist through the GPT era; left in 2024 to found **Safe Superintelligence** (SSI) - still research-only, no product, by choice.
- ▶ **Alex Krizhevsky** - the one who actually trained it - quietly stepped away from the field.

One paper, three completely different lives. There is no single "what happens after you change a field".

Vision in 2026 (I): understanding

Two shifts since this lecture's timeline ended:

1. **Attention ate the frontier.** Vision Transformers (ViT) treat an image as a *sequence of patches* - no convolution at all. (L19 closes this thought against CNN inductive bias.)
2. **Image-text pretraining** (CLIP, 2021) fused vision with language: today's assistants (GPT, Gemini, Claude) see - show them a photo, ask questions.

Classification stopped being a standalone task and became a **capability inside larger systems**.

► try it: show a VLM a photo, ask it questions

Vision in 2026 (II): generation

Diffusion models flipped the task: classification asks "what is this?" - diffusion answers "*draw me one*".

- ▶ Images: Stable Diffusion, DALL-E, Flux. Video followed: Sora, Veo.
- ▶ Intuition in one line: learn to **undo noise, step by step** - the mechanics belong to the GenAI chapter.
- ▶ And foundation models took the classic tasks: **SAM** segments anything, promptable, no task-specific training.

Generative vision gets its own treatment in the **GenAI chapter** - here it is context, not curriculum.

▶ SAM demo: click an object, get its mask

And yet: CNNs did not die

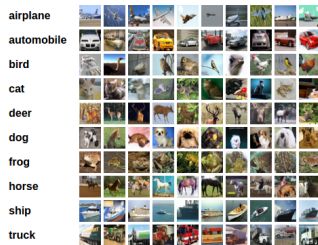
On phones, cars, cameras and sorting lines - including our **pomegranate belt** - CNNs and CNN hybrids remain the workhorse:

- ▶ **cheap and fast** at the edge, where there is no data center;
- ▶ the **right inductive bias** when data is limited (L16's whole argument);
- ▶ ResNets are still the **default baseline** in papers, a decade on.

You learned the layer that still does most of the world's looking.

Recap

- ▶ A shared **benchmark** (ImageNet) turned architecture design into measurable progress.
- ▶ **LeNet** wrote the blueprint; **AlexNet** = L15's tricks at GPU scale; **VGG** = small filters, stacked deep.
- ▶ Deeper plain nets *degrade* → **residual learning** (= boosting in depth); skips also carry gradients.
- ▶ conv-BN-ReLU blocks, 1×1 convs, **GAP** instead of a giant dense head.
- ▶ **Load pretrained models**; don't retype architectures.
- ▶ The arc: Bell Labs checks → ImageNet → ViT / VLMs / diffusion - and CNNs still run the edge.



CIFAR-10 - HW2's arena: small custom CNN vs a ResNet-18-scale net.

Next: these models were trained on 1.2M images. You have 2,000. Transfer learning - stand on ImageNet's shoulders (L18).